

Memoria de Práctica 2:

DOCKER

Autor: Alfonso Ramos Rojas

DockerHub: <https://hub.docker.com/u/alraro>

1. Introducción y Objetivos

Este proyecto desarrolla una arquitectura de microservicios resiliente para la monitorización industrial mediante contenedores Docker. El sistema simula la ingesta, almacenamiento y análisis de datos de sensores en tiempo real, aplicando principios de **Ingeniería de Software Crítico**.

Los **objetivos principales** abordados son:

1. **Arquitectura Modular:** Orquestación de servicios independientes para la lógica de negocio (API REST en Flask), almacenamiento y visualización (Grafana), facilitando la escalabilidad horizontal con docker swarm.
2. **Monitorización Especializada:** Implementación de persistencia eficiente mediante **RedisTimeSeries**, una base de datos en memoria optimizada para series temporales.
3. **Inteligencia Artificial:** Integración de un modelo de Deep Learning (Autoencoder LSTM) para la detección automatizada de anomalías en el flujo de datos.
4. **Alta Disponibilidad (HA):** Eliminación de puntos únicos de fallo mediante el despliegue y validación de dos arquitecturas distribuidas:
 - **Redis Sentinel:** Para garantizar la replicación y el *failover* automático.
 - **Redis Cluster:** Para implementar *sharding* (fragmentación) y algoritmos de consenso distribuido.

2. Parte 1: Arquitectura Base y Monitorización

En esta fase inicial, se ha establecido la infraestructura fundamental del sistema utilizando contenedores Docker.

2.1. Componentes del Sistema

El sistema se articula sobre cuatro pilares tecnológicos, cada uno encapsulado en su propio contenedor:

- **API Web (Servicio **web**):** Desarrollada en Python utilizando el *microframework* Flask. Actúa como la capa de lógica de negocio y punto de entrada para los sensores.
- **Almacenamiento (RedisTimeSeries):** Para la persistencia de datos se ha usado Redis TimeSeries, módulo de redis para almacenar datos de series temporales..

- **Visualización (Grafana):** Permite consultar directamente la evolución de las métricas (temperatura) en tiempo real mediante un dashboard conectado a Redis.
- **Infraestructura (Docker):** La orquestación se gestiona mediante Docker Compose, definiendo redes virtuales ([webnet](#)) y volúmenes persistentes ([grafana_data](#)) para garantizar que la configuración de visualización sobreviva a los reinicios de los contenedores.

2.2. Implementación de la API REST

La aplicación [app.py](#) expone una interfaz REST simplificada para la interacción con los dispositivos IoT.

- **Endpoint de inserción de datos (/nuevo?dato):**
Este endpoint recibe mediciones a través del parámetro dato. Internamente, valida la entrada y utiliza el comando TS.ADD de Redis para insertar el par (timestamp, valor) en la serie temporal *temperature*. La gestión de errores asegura que fallos en la conexión con Redis sean reportados adecuadamente al cliente.
- **Endpoint de Consulta (/listar):**
Permite recuperar el histórico de mediciones almacenadas utilizando el comando TS.RANGE (o TS.REVRANGE para orden inverso), junto al hostname de la máquina que las devuelve, con el fin de comprobar el funcionamiento con **Docker Swarm**.

2.3. Despliegue y Pruebas

El despliegue de esta arquitectura base se define en el archivo [docker-compose-standard.yml](#). Se han configurado restricciones de despliegue ([constraints: \[node.role == manager\]](#)) y políticas de reinicio ([restart: on-failure](#)) para simular un entorno de producción robusto.

Verificación de Persistencia y Visualización:

Tras levantar el stack, se procedió a la inyección de datos sintéticos mediante el script de pruebas. Se configuró un DataSource de Redis en Grafana y se creó un panel para visualizar la serie temporal *temperature*.

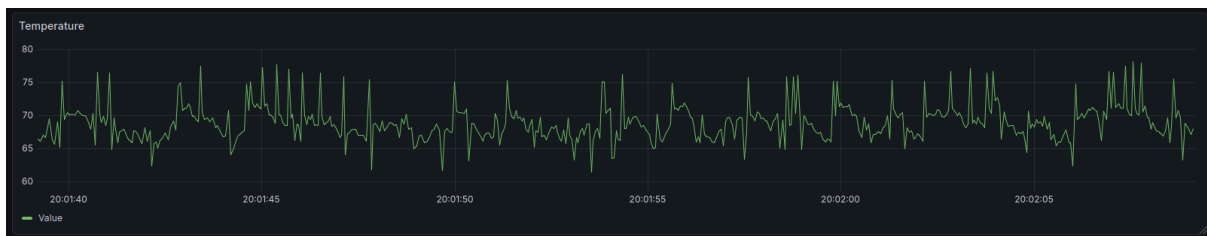


Fig 1: Gráfica en Grafana mostrando la evolución temporal de los datos de temperatura introducidos a través de la API.

Ejecutamos el sistema con 5 réplicas en docker swarm, el cual balancea la carga entre las 5.

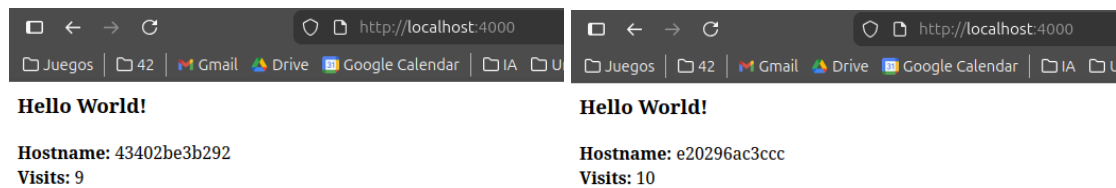


Fig 2-3: Capturas del endpoint base balanceando la carga

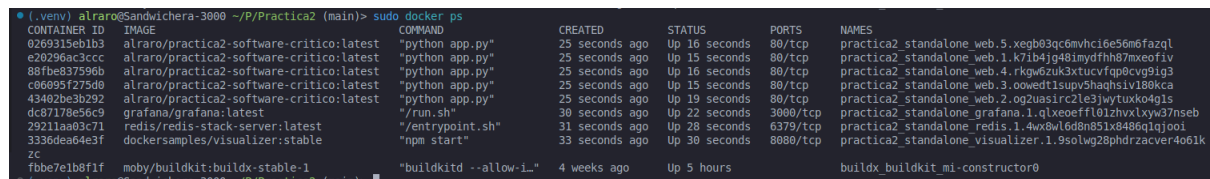


Fig 4: Captura de todos los servicios de docker swarm

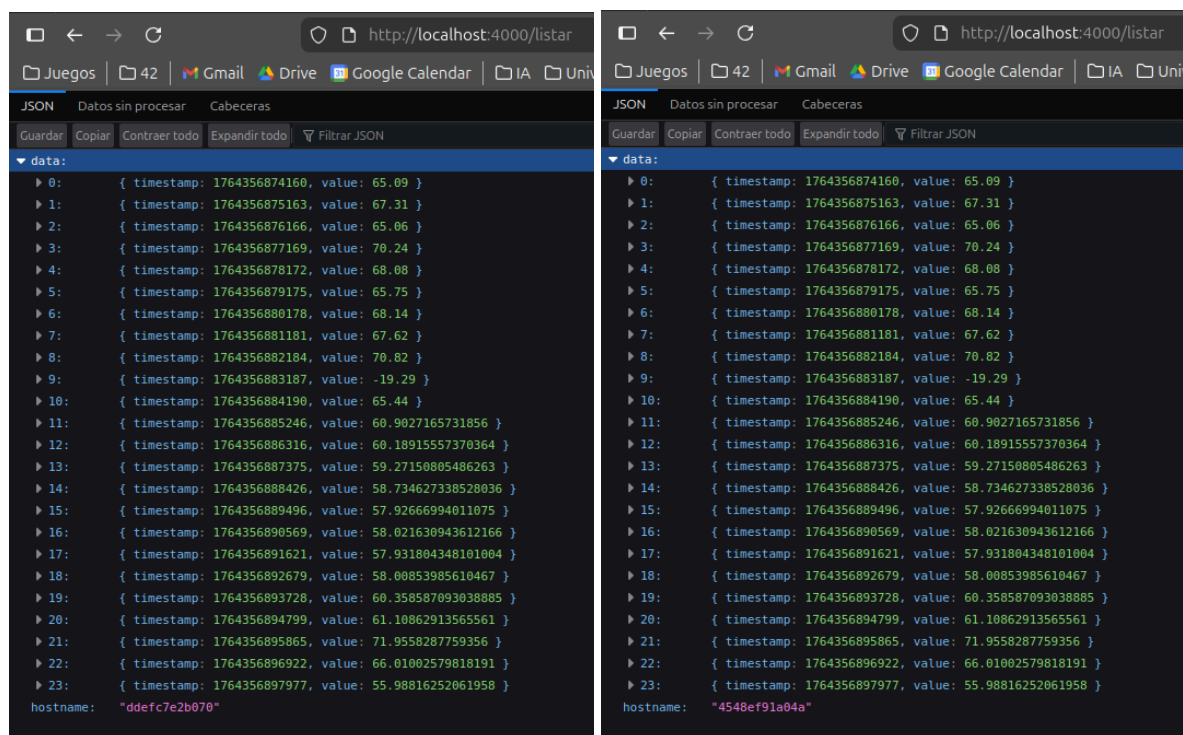


Fig 5-6: Captura de múltiples peticiones a /listar donde se observa el balance de carga.

3. Parte 2: Detección de Anomalías (Deep Learning)

3.1. Modelo LSTM Autoencoder

Para la detección de anomalías he usado el modelo de **LSTM Autoencoder** creado en la práctica 1, junto al **Min Max Scaler** usado en la misma, ambos importados en la aplicación en los archivos `lstm_autoencoder_model.keras` y `scaler.plk`.

3.2. Lógica del Endpoint `/detectar`

1. Construcción de la Ventana Deslizante:

Al recibir un nuevo dato, el sistema consulta en Redis las 11 últimas mediciones almacenadas. Estos datos históricos se combinan con el valor actual para formar una ventana de contexto de 12 pasos (`window_size = 12`).

2. Inferencia y Cálculo del Error:

La ventana procesada se introduce en el Autoencoder. El modelo intenta reconstruir la entrada. Posteriormente, se calcula el Error Absoluto Medio (MAE) entre la ventana original y la reconstruida.

3. Umbral de Decisión:

Se ha definido un umbral estático de `threshold = 0.0388` en la práctica 1. Si el MAE calculado supera este valor, el sistema etiqueta la medición con "anomalía": "sí".

4. Estrategia de Sanitización (Persistencia Inteligente):

Para proteger la calidad de los datos históricos, se ha implementado una lógica de persistencia condicional:

- **Caso Normal:** Se almacena el dato crudo recibido del sensor.
- **Caso Anómalo:** Se descarta el dato crudo y se almacena en su lugar el **valor reconstruido** por el modelo. Esta técnica de "sanitización" evita que un valor extremo corrompa la ventana deslizante para las siguientes 11 predicciones, estabilizando el sistema ante fallos transitorios del sensor.

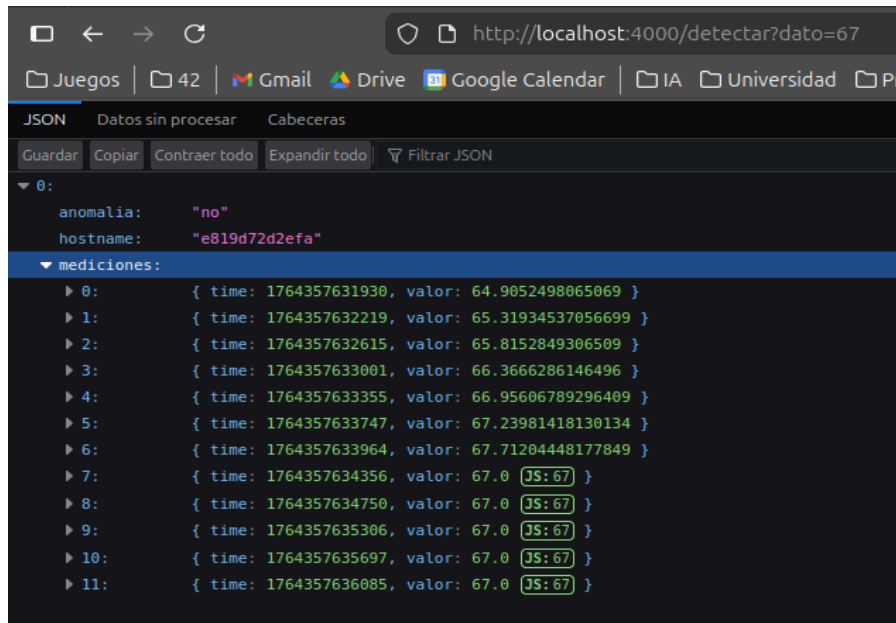


Fig 7: Captura de una petición al endpoint `/detectar?dato` con un valor normal donde se observa la detección del modelo correcta

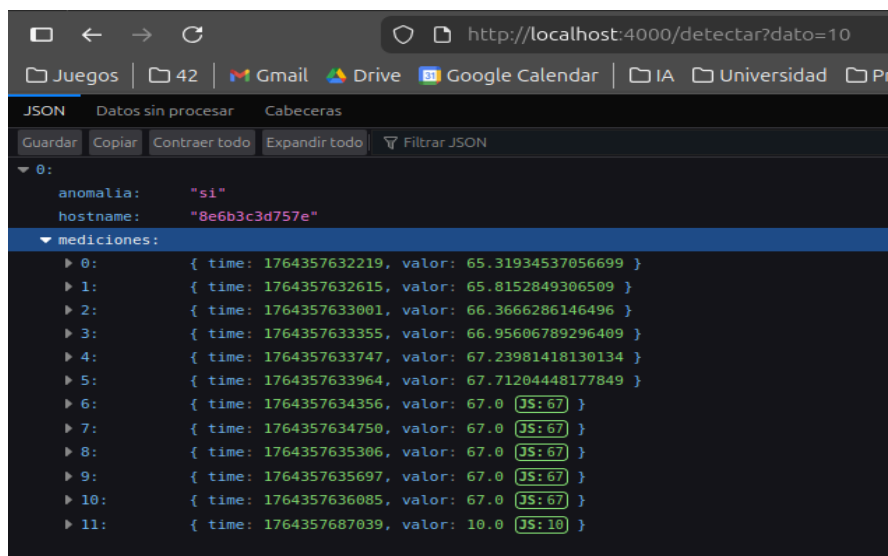


Fig 8: Captura de una petición al endpoint `/detectar?dato` con un valor anómalo donde se observa la detección del modelo correcta

4. Parte 3: Alta Disponibilidad y Tolerancia a Fallos

Se desplegaron y validaron dos arquitecturas distribuidas distintas: **Redis Sentinel** (enfocada en alta disponibilidad) y **Redis Cluster** (enfocada en escalabilidad horizontal y disponibilidad).

4.1. Opción A1: Arquitectura Redis Sentinel

Proporciona monitorización, notificaciones y recuperación automática (*failover*). En el siguiente **video demostración** se muestra el funcionamiento de los centinelas ante un fallo del sistema: <https://youtu.be/tbHkthJR57A>

4.1.1. Diseño de la Infraestructura

La topología desplegada en [docker-compose-redis-sentinel.yml](#) consta de los siguientes elementos interconectados en la red [webnet](#):

- **Nodo Maestro:** 1 Maestro ([redis-master](#))
- **Nodos Réplicas:** 2 Réplicas del Maestro ([replica-1](#), [replica-2](#))
- **Nodos Centinelas:** 3 instancias de Sentinel ([sentinel-1](#), [sentinel-2](#), [sentinel-3](#)), con un quorum de 2, es decir que al menos 2 de los 3 centinelas deben de estar de acuerdo para cambiar de maestro.

4.1.2. Desafíos Técnicos y Soluciones

Durante la implementación, surgieron varios desafíos críticos relacionados con la contenerización de sistemas distribuidos:

1. Problema de Versiones e Incompatibilidad de Módulos:

- **Problema:** Inicialmente, los contenedores Sentinel fallaban al intentar cargar el módulo [RedisTimeSeries](#).
- **Causa:** Los centinelas no requieren almacenar series temporales, solo gestionar la configuración. Sin embargo, al compartir configuración con los nodos de datos, intentaban cargar una librería inexistente en su imagen base.
- **Solución:** Se segregaron las imágenes. Los nodos de datos utilizan [redis/redis-stack-server](#) (que incluye los módulos necesarios para la API), mientras que los centinelas utilizan la imagen ligera [redis:6.2](#) sin la directiva [--loadmodule](#), optimizando el consumo de recursos.

2. Seguridad de Red (Protected Mode):

- **Problema:** Tras el despliegue, los centinelas detectaban a las réplicas pero no lograban conectar con ellas, marcándolas inmediatamente como [s_down](#) (Subjectively Down).
- **Solución:** Redis, por defecto, impide conexiones externas sin contraseña en modo protegido. Se añadió explícitamente el flag [--protected-mode=no](#) en el comando de arranque de todos los nodos ([redis-master](#), [redis-replica-x](#)) para

permitir la comunicación libre dentro de la red privada de Docker.

3. Persistencia de Configuración en Docker:

- **Problema:** Sentinel necesita reescribir su archivo `sentinel.conf` tras un *failover* para persistir la nueva topología. Al montar este archivo desde el host, surgían errores de permisos o bloqueos de escritura ("Device or resource busy").
- **Solución:** Se implementó una generación dinámica de la configuración. En lugar de montar un archivo externo, el comando de inicio del contenedor genera el fichero `sentinel.conf` al vuelo en el directorio `/tmp/` mediante comandos `echo`. Esto garantiza que cada despliegue comience con una configuración limpia y tenga permisos de escritura nativos dentro del contenedor.

4.1.3. Pruebas de Failover (Simulación de Fallo)

Para validar la resiliencia del sistema, se realizaron pruebas de interrupción del servicio tal y como se muestran detalladamente en el **video de demostración**. Tiramos el contenedor del maestro y vemos como los centinelas votan para elegir un nuevo maestro hasta que lo consiguen.

```
redis-sentinel-1 | 10:X 28 Nov 2025 20:17:57.755 # +switch-master mymaster 172.22.0.2 6379 172.22.0.6 6379
redis-sentinel-1 | 10:X 28 Nov 2025 20:17:57.755 * +slave slave 172.22.0.5:6379 172.22.0.5 6379 @ mymaster 172.22.0.6 6379
redis-sentinel-1 | 10:X 28 Nov 2025 20:17:57.755 * +slave slave 172.22.0.2:6379 172.22.0.2 6379 @ mymaster 172.22.0.6 6379
redis-sentinel-3 | 10:X 28 Nov 2025 20:17:57.821 # +config-update-from sentinel 775019505825f54f101233570d6336bc64c4d956 172.22.0.2 6379
redis-sentinel-3 | 10:X 28 Nov 2025 20:17:57.821 # +switch-master mymaster 172.22.0.2 6379 172.22.0.6 6379
redis-sentinel-2 | 10:X 28 Nov 2025 20:17:57.821 # +config-update-from sentinel 775019505825f54f101233570d6336bc64c4d956 172.22.0.2 6379
redis-sentinel-2 | 10:X 28 Nov 2025 20:17:57.821 # +switch-master mymaster 172.22.0.2 6379 172.22.0.6 6379
redis-sentinel-2 | 10:X 28 Nov 2025 20:17:57.821 * +slave slave 172.22.0.5:6379 172.22.0.5 6379 @ mymaster 172.22.0.6 6379
```

Fig 9: Secuencia de logs donde el Sentinel detecta la caída del nodo 172.22.0.2 y promueve a la réplica 172.22.0.6.

4.2. Opción A2: Arquitectura Redis Cluster

Como alternativa para escenarios de alta demanda, se implementó un **Redis Cluster**, que introduce el concepto de *sharding* (particionado de datos) además de la alta disponibilidad. Podemos ver el funcionamiento detallado del sistema y pruebas realizadas en el siguiente **video demostración**: <https://youtu.be/wTkep2tNfRg>

4.2.1. Diseño de la Infraestructura (Sharding)

Se configuró un cluster completo de **6 nodos** definidos en `docker-compose-redis-cluster.yml`:

- **3 Maestros:** Se reparten los 16384 *hash slots* del espacio de claves.
- **3 Réplicas:** Cada maestro tiene asignada una réplica directa para tolerancia a fallos.
- **Red:** Se utilizó una subred personalizada (`172.28.0.0/24`) con IPs estáticas para garantizar una formación determinista del cluster durante el arranque.

4.2.2. Implementación del Cliente

La integración con la API REST requirió actualizaciones significativas en `app.py`:

- **Librería Nativa:** Se descartó la librería obsoleta `redis-py-cluster` sugerida en documentación antigua. Se utilizó la clase nativa `RedisCluster` incluida en las versiones modernas de `redis-py`.
- **Gestión de Nodos:** El cliente se inicializa recibiendo una lista de nodos semilla (`startup_nodes`) definidos en la variable de entorno `REDIS_CLUSTER_NODES`. Se utilizan objetos `ClusterNode` para una gestión tipada correcta.
- **Decodificación:** Se configuró `decode_responses=False` para mantener la compatibilidad con el código de procesamiento de datos existente, que espera recibir bytes en lugar de strings.

4.2.3. Inicialización Robusta

La creación de un cluster requiere que los nodos estén operativos antes de unirlos. Para evitar condiciones de carrera (*race conditions*), se sustituyeron los tiempos de espera arbitrarios (`sleep`) por **Healthchecks** nativos de Docker.

- El contenedor efímero `redis-cluster-init` tiene una dependencia estricta (`condition: service_healthy`) sobre los 6 nodos. Solo ejecuta el comando de creación (`redis-cli --cluster create ...`) cuando todos los nodos responden afirmativamente al `PING`.

4.2.4. Pruebas de Consenso y Quórum

Se validaron los límites teóricos del algoritmo de consenso del Cluster:

- **Rolling Failover:** Al pausar un único maestro, su réplica detecta la ausencia, inicia una votación y obtiene la mayoría necesaria de los otros maestros para ascender. El servicio continúa sin interrupción.
- **Pérdida de Quórum:** Se comprobó que al pausar simultáneamente los 3 maestros, el cluster entra en estado de fallo y detiene el servicio. Esto es el comportamiento correcto y esperado, ya que las réplicas no pueden alcanzar una mayoría absoluta de votos (quórum) para promocionarse, priorizando la consistencia de los datos sobre la disponibilidad (Teorema CAP).

```
Cada 1,0s: sudo docker exec -it redis-node-2 redis-cli -c cluster nodes | sort
Sandwichera-3000: Fri Nov 28 22:02:58 2025
254c7919ea995b5f26f070f1a1eae916452f8d9 172.28.0.14:6379@16379 slave 4b7981542ca1a17ea54425071fa12228c2479b42 0 1764363778594 3 connected
48cb9357f2ee5d467342a1919479cfbbf1cd97e 172.28.0.16:6379@16379 slave e80c5edd0087063bdb1e84fd4137172aa0058448 0 1764363777185 2 connected
4b7981542ca1a17ea54425071fa12228c2479b42 172.28.0.13:6379@16379 master - 0 1764363777000 3 connected 10923-16383
84004fa23a45ce341d071edebe15b5725f848a3d 172.28.0.15:6379@16379 master - 0 1764363778192 7 connected 0-5460e
9b51a876393e3aad18cca6419248c54177cb12d 172.28.0.11:6379@16379 slave 84004fa23a45ce341d071edebe15b5725f848a3d 0 1764363777000 7 connected
e80c5edd0087063bdb1e84fd4137172aa0058448 172.28.0.12:6379@16379 myself,master - 0 0 2 connected 5461-109228
```

Fig 10: Salida del comando CLI mostrando la distribución de slots y el estado de los nodos tras un failover del nodo 3.

En el **video demostración** hay una se muestra todo el funcionamiento de forma más detallada

5. Google Cloud

Para complementar a la práctica, he creado una instancia en google cloud con ubuntu donde he alojado el sistema.

☐ Estado	Nombre ↑	Zona	Recomendaciones	En uso por	IP interna	IP externa	Conectar
☒	instance-20251027-082657	us-central1-c			10.128.0.2 (nic0)	34.31.82.46 (nic0)	SSH ▾

Fig 11: Instancia de Google Cloud

Para la conexión con la misma ha sido necesaria la creación de una regla en el firewall de la instancia que permita el acceso a los puertos 3000 y 4000 para permitir las peticiones a Grafana y a la aplicación respectivamente.

regla- conexion- docker	1	Global	1000	—	Entrada	Rangos de...	—	tcp:3000, 4000	Permitir
---	---	--------	------	---	---------	--------------	---	----------------	----------

Fig 12: Regla de conexión en el firewall.

6. Script de Validación Automatizada ([tester.py](#))

Para comprobar el funcionamiento normal de la aplicación, he creado un tester de python que manda peticiones con valores normales cada intervalo de tiempo, y de forma aleatoria manda anomalías para probar la detección de las mismas.

7. Anexo: Vídeos e Instrucciones de Despliegue

En el este apartado se muestran los videos de demostración y las instrucciones de despliegue de cada uno de los sistemas (standalone, sentinel y cluster).

7.1 Vídeos de demostración

- Demostración con Redis Sentinel: <https://youtu.be/tbHkthJR57A>
- Demostración Grafana: <https://youtu.be/QNM5gG9q4X0>
- Demostración con Redis Cluster: <https://youtu.be/wTkep2tNfRg>

7.2 Docker

- Enlaces de Hub Docker:
 - Perfil: <https://hub.docker.com/u/alraro>
 - Imagen: <https://hub.docker.com/r/alraro/practica2-software-critico>
- Imagen de docker completa:
 - **alraro/practica2-software-critico:1.1.14**

- **alraro/practica2-software-critico:latest**

7.3 Instrucciones de despliegue

Tener en el mismo directorio todos los archivos entregados y realizar el comando pertinente para lanzar cada uno de los servicios.

7.3.1 Sistema estándar

- `sudo docker swarm init`
- `sudo docker stack deploy -c docker-compose-standard.yml practica2_standalone`

7.3.2 Sistema con Redis Sentinel

- `sudo docker compose -f docker-compose-redis-sentinel.yml up -d`

7.3.3 Sistema con Redis Cluster

- `sudo docker compose docker-compose-redis-cluster.yml up -d`